JASP Student Developers

Creating Plots in JASP

04.03.2026

julian.wuth@student.uva.nl

Contents

1. Quick recap
2. Plots
   - Coding challenge

Contents

1. Quick recap
2. Plots
   - Coding challenge

```r
BinomialTest <- function(jaspResults, dataset, options) {
```

options

options	list [11]	List of length 11
.meta	list [3]	List of length 3
checkedFactor	character [1]	''
nominalOrOrdinalVa...	character [1]	''
nominalOrOrdinalVa...	character [1]	'unknown'
plotHeight	integer [1]	320
plotWidth	integer [1]	480
ts	character [1]	''
ts.types	list [0]	List of length 0
values	list [0]	List of length 0
xs	character [1]	'JaspColumn_15_Encoded'
xs.types	character [1]	'scale'

dataset

	JaspColumn_15_Encoded
1	0.99122228
2	-1.95334497
3	1.57630164
4	0.94793341
5	-0.53255250
6	-1.36359763
7	-0.65497938
8	-0.75144595
9	0.31018152
10	-0.67527073

```r
.ttestIndependentMainTable <- function(jaspResults, dataset, options, ready, type) {
  if (!is.null(jaspResults[["ttest"]]))
    return()

  optionsList <- .ttestOptionsList(options, type)

  # Create table
  ttest <- createJaspTable(title = gettext("Independent Samples T-Test"))
```

Contents

1. Quick recap
2. Plots
   - Coding challenge

Plotting in JASP

Step-by-step

1. Create the plot
2. Add plot to output
3. Fill the plot

Plotting in JASP

R-code (steps 1&2)

```
.binomPlotDescriptives <- function(jaspResults, dataset, options, ready) {
  binomialPlot <- createJaspPlot(title = "Descriptives",  width = 160, height = 320)

  binomialPlot$dependOn(c("variables", "testValue", "descriptivesPlotsConfidenceInterval"))

  binomialPlot$addCitation("JASP Team (2018). JASP (Version 0.9.2) [Computer software].")

  jaspResults[["binomialPlot"]] <- binomialPlot

  if (!ready)
    return()
```

Plotting in JASP

R-code (step 3)

- Like with tables, filling is usually handled by a separate function
- The final plot is assigned to the plotObject property

```
.binomFillPlotDescriptives <- function(binomialPlot, dataset, options)

  plot <- ggplot2::ggplot(<code>)

  binomialPlot$plotObject <- plot

  return()
}
```

Plotting in JASP

Usage of filling function (step 3)

- Filling function is then called within the function that creates the plot if analysis is ready (cf. output tables)

```
.binomPlotDescriptives <- function(jaspResults, dataset, options, ready) {
  binomialPlot <- createJaspPlot(title = "Descriptives",  width = 160, height = 320)

  binomialPlot$dependOn(c("variables", "testValue", "descriptivesPlotsConfidenceInterval"))

  binomialPlot$addCitation("JASP Team (2018). JASP (Version 0.9.2) [Computer software].")

  jaspResults[["binomialPlot"]] <- binomialPlot

  if (!ready)
    return()

  .binomFillPlotDescriptives(binomialPlot, dataset, options)

  return()
}
```

jaspGraphs

JASP's own plotting package

- Mainly used for applying JASP style to ggplot objects:

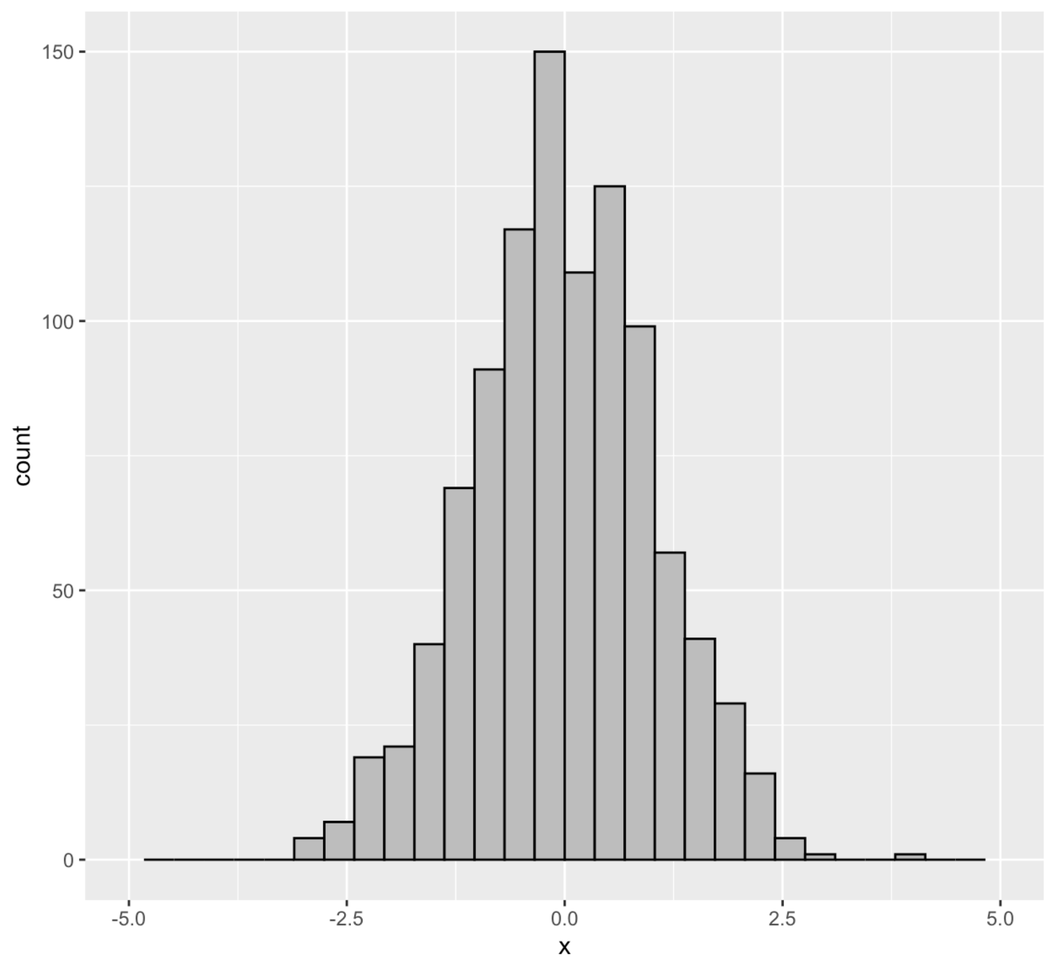
```
plot +
  jaspGraphs::geom_rangeframe() + # add lines on the x-axis and y-axis
  jaspGraphs::themeJaspRaw()      # add the JASP theme
```

- Also comes with complete plotting functions (e.g., `jaspHistogram()`)
- Only available on GitHub. E.g., via `remotes::install_github("jasp-stats/jaspGraphs")`
- For a complete overview, see here.

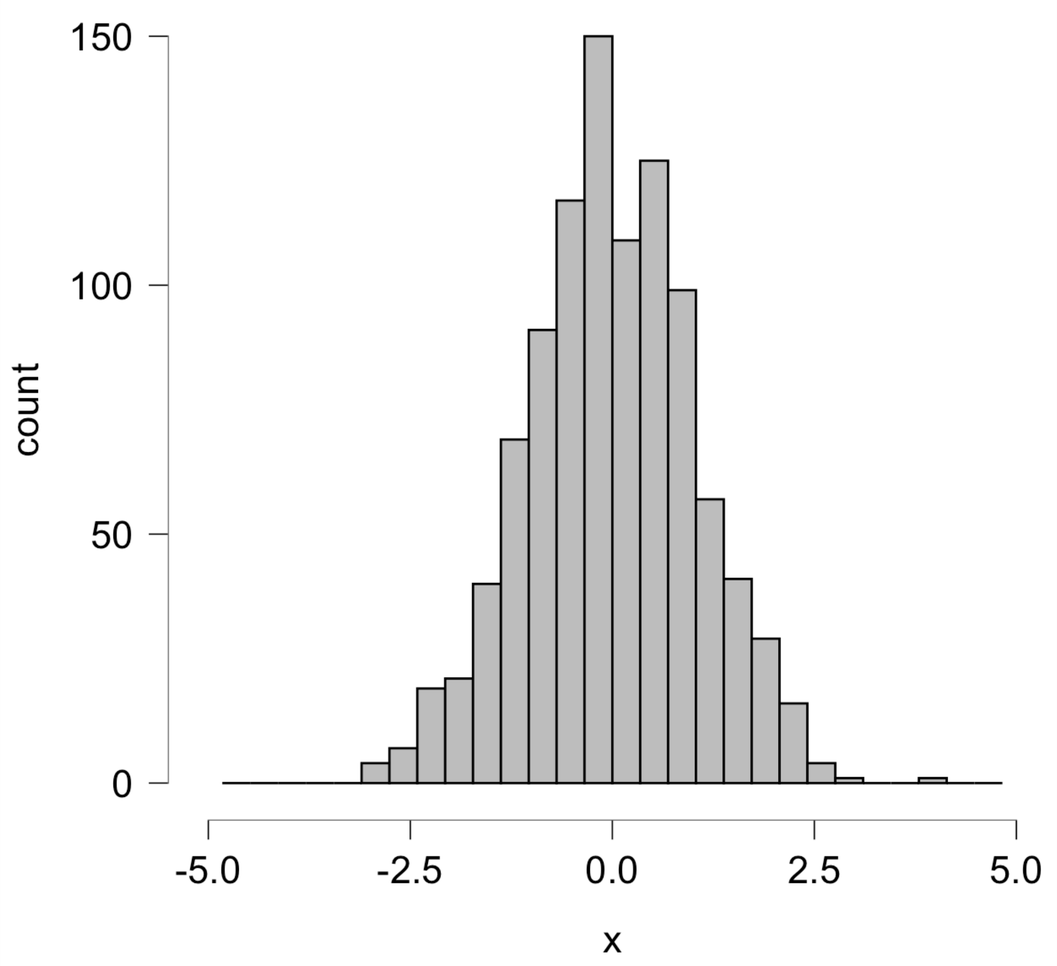
jaspGraphs

Example styling of a ggplot

Before



After



Coding challenge

Creating a histogram in JASP

- Clone this repository.
- Go to *R/histogram_skeleton.R*
- The file contains several TODOs for you to work on
- If you need more information about creating plots in JASP, consult the documentation.
- Two potential solutions can be found in *R/histogram_jaspGraphs.R* or *R/histogram_solution.R*

See you next time!